Ejercicio N1: Modelo de Zoo del Mundo

En MongoDB, la estrategia clave es **la desnormalización y el anidamiento (embedding)**.

Una propuesta válida de base de datos para "todos los zoológicos del mundo", estructurada en colecciones principales que agrupan los datos más frecuentemente consultados juntos.

Principios de Diseño para la Propuesta en MongoDB:

1. **Colecciones Principales:** Tendremos dos colecciones principales: `zoos` (para la información de los zoológicos y su ubicación) y `animales` (para los individuos).
2. **Anidamiento de Ubicación:** En lugar de tener colecciones separadas para `Ciudad`, `Pais` y `Continente`, anidaremos esta información directamente dentro del documento del zoológico para que, al buscar un zoológico, obtengamos toda su ubicación en una sola consulta.
3. **Anidamiento de Especie:** En lugar de tener una colección separada para `EspecieAnimal`, anidaremos esta información detallada directamente dentro del documento del animal.
4. **Referencias Simplificadas:** Para conectar animales a zoológicos y países de origen, usaremos referencias simples (IDs), pero también mantendremos algunos datos desnormalizados (como el nombre del zoológico o el nombre de la especie en el animal) para búsquedas rápidas sin necesidad de "joins" complejos (usando `$lookup` solo cuando sea estrictamente necesario para reportes agregados).

Propuesta de Estructura de Colecciones y JSONs de Ejemplo:

Colección 1: `zoos`

Esta colección almacena la información de cada zoológico y su jerarquía geográfica completa. Es la fuente de verdad para los datos del zoológico y su ubicación.

Estructura: Cada documento representa un zoológico. La ubicación se anida de forma jerárquica.

Ejemplo de un documento JSON en la colección `zoos`:

JSON

```
{
  "_id": ObjectId("65e6...0001"), // ID único de MongoDB
  "nombre": "Zoológico de Chapultepec",
  "tamaño": "17 hectáreas",
  "presupuesto": 15000000.00, // En moneda local, por ejemplo
  "ubicacion": {
    "ciudad": "Ciudad de México",
    "pais": {
      "nombre": "México",
      "continente": {
        "nombre": "América del Norte"
      }
    }
  }
}
```

Explicación del cambio: Hemos eliminado las tablas `Ciudad`, `Pais` y `Continente`. Al anidar la jerarquía geográfica completa dentro de cada zoológico, una sola consulta como `db.zoos.find({"ubicacion.pais.nombre": "México"})` nos devuelve todos los zoológicos de México instantáneamente, sin tener que consultar tres tablas diferentes.

Colección 2: `animales`

Esta colección almacena la información detallada de cada animal individual. Incluye su información biológica y su estado actual.

Estructura: Cada documento representa un animal individual. La información de la especie está anidada. Hay referencias (IDs y nombres desnormalizados) para el zoológico y el país de origen.

Ejemplo de un documento JSON en la colección `animales`:

JSON

```
{
  "_id": ObjectId("65e6...0101"), // ID único del animal (reemplaza a 'numeroIder
  "sexo": "Hembra",
  "añoNacimiento": 2018,

  // Información de la Especie (Anidada)
  "especie": {
    "nombreVulgar": "Panda Gigante",
    "nombreCientifico": "Ailuropoda melanoleuca",
    "peligroExtincion": true,
    "familia": "Ursidae" // Desnormalizado de la tabla Familia
  },

  // Referencia al Zoológico donde se encuentra (Fuente de Verdad es _id)
  "zoo": {
    "id": ObjectId("65e6...0001"), // ID del documento en la colección 'zoos'
    "nombre": "Zoológico de Chapultepec" // Nombre desnormalizado para visualiza
  },

  // Referencia al País de Origen de la especie
  "paisOrigen": {
    "nombre": "China" // Desnormalizado para búsquedas
  }
}
```

Explicación del cambio:

1. **Especie Anidada:** La información de `EspecieAnimal` (nombre vulgar, científico, peligro de extinción) y `Familia` se anida dentro de cada animal. Esto significa que cuando consultas un animal, obtienes toda su información biográfica de una vez, sin ir a las tablas de especies o familias.
2. **Referencia de Zoológico con Desnormalización:** Guardamos el `id` del zoológico (`zoo.id`) para la integridad, pero también guardamos el `nombre` (`zoo.nombre`). Esto nos permite mostrar el nombre del zoológico del animal en un listado rápido sin tener que consultar la colección `zoos`. Si el zoológico cambia de nombre, solo necesitamos actualizar la colección `zoos`, y el nombre desnormalizado en `animales` se puede actualizar en segundo plano si es crítico.
3. **Referencia de País Simplificada:** Similar al zoológico, guardamos el nombre del país de origen de la especie para búsquedas rápidas (por ejemplo, "todos los animales originarios de China").

Beneficios de esta Propuesta en MongoDB:

1. **Lecturas de Alto Rendimiento:** Consultar un zoológico o un animal individual requiere una sola operación de lectura, obteniendo todos los datos relacionados de forma instantánea. No hay "joins" en tiempo de consulta para operaciones comunes.

2. **Agrupación Lógica de Datos:** Los datos geográficos están donde se necesitan (con el zoológico). Los datos biológicos están donde se necesitan (con el animal).
3. **Flexibilidad de Esquema:** Si en el futuro un zoológico quiere añadir "horarios de apertura" o un animal necesita un historial médico, es muy fácil añadir esos campos como nuevos subdocumentos en las colecciones existentes, sin alterar un esquema rígido.
4. **Escalabilidad:** Al reducir las dependencias cruzadas entre colecciones, es más fácil distribuir las colecciones en diferentes servidores (sharding) a medida que la base de datos crece para cubrir "todos los zoológicos del mundo".

Escenario de Migración

... si solo necesito cargar los continentes en una lista de valores en la web asimismo cargar todos los países de un continente y asimismo cargar ciudades de un país en particular.... ¿no debería ir a buscarlos a la colección zoo cierto?

En el caso de uso específico (llenar **Selects** o **Dropdowns** dinámicos en cascada), ir a la colección `zoos` sería ineficiente y complicado.

Aquí es donde entra el concepto de **Colecciones de Soporte (Lookups)**. Aunque para el "perfil" del zoológico anidamos los datos, para la interfaz de usuario (UI) de búsqueda o filtrado necesitamos una estructura optimizada para diccionarios.

La Propuesta: Una Colección Maestro de Geografía

En lugar de tener tres colecciones separadas (`continente`, `pais`, `ciudad`), en MongoDB se recomienda consolidarlas en una sola colección llamada `geografia` o `localidades`. Esto te permite traer toda la jerarquía de un país en una sola consulta.

Opción A: Colección Única `localidades` (Recomendada para Web)

Este modelo permite que, cuando el usuario selecciona un país, ya tengas todas sus ciudades disponibles sin volver a consultar la base de datos.

JSON

```
[
  {
    "_id": "cont_america_sur",
    "tipo": "continente",
    "nombre": "América del Sur",
    "paises": [
      {
        "nombre": "Chile",
        "ciudades": ["Santiago", "Puente Alto", "Buin", "Concepción"]
      },
      {
        "nombre": "Argentina",
        "ciudades": ["Buenos Aires", "Mendoza", "Córdoba"]
      }
    ]
  },
  {
    "_id": "cont_europa",
    "tipo": "continente",
    "nombre": "Europa",
    "paises": [
      {
        "nombre": "España",
        "ciudades": ["Madrid", "Barcelona", "Valencia"]
      }
    ]
  }
]
```

Ventaja para tu App Web: * **Paso 1:** Cargos los continentes: `db.localidades.find({}, {nombre: 1}).`

- **Paso 2:** El usuario elige "América del Sur". Tu backend envía el documento completo de ese continente.
- **Paso 3:** La lógica de tu Frontend (JavaScript) ya tiene la lista de países y ciudades en memoria, por lo que los siguientes dropdowns cargan **instantáneamente** sin latencia de red.

¿Y qué pasa con Especie y Familia?

Aplica la misma lógica. Si necesitas un buscador donde el usuario primero elija la Familia (ej. *Felidae*) y luego la Especie (ej. *León*), crea una colección maestro llamada `taxonomia`.

Colección `taxonomia`:

JSON

```
{
  "_id": "fam_felidae",
  "familia": "Felidae",
  "especies": [
    { "nombreVulgar": "León", "nombreCientifico": "Panthera leo" },
    { "nombreVulgar": "Tigre", "nombreCientifico": "Panthera tigris" }
  ]
]
```

Resumen de la Estrategia de Migración

Para que tu sistema sea profesional y rápido, debes usar un **modelo híbrido**:

1. **Colecciones Operativas (`zoos`, `animales`):** Aquí los datos están **anidados y desnormalizados**. Se usan para mostrar la información final, fichas técnicas y resultados de búsqueda.
2. **Colecciones de Diccionario (`localidades`, `taxonomia`):** Aquí los datos están **agrupados por jerarquía**. Se usan exclusivamente para alimentar los componentes de la interfaz (Selects, Autocompletados, Filtros).

¿Por qué este modelo es mejor que el relacional puro? En SQL, para cargar las ciudades de un continente tendrías que hacer 2 JOINs. En este modelo de MongoDB, haces **una sola consulta** por ID y obtienes el árbol completo de opciones para el usuario.